Earn Google Badge

Basics

Fundamentals

Scalability

Databases in Designing Systems

High Level Design(HLD)

Low Level Design(LLD)

Design Patterns

System Design Cours

Explore

System Design of Splitwise Backend (Expense Sharing Apps)

Last Updated : 12 Jul, 2024

In this post, we will look into the System Design of Expense Sharing Applications such as Splitiwse. We will deep dive into the LLD of Splitwise, the Happy Flow of Splitwise, and also how to design the backend for Splitwise or such Expense Sharing Applications.

Table of Content

- About Expense-Sharing Applications
- System Design of Splitwise
- Step 1: Defining Happy Flow for Splitwise
- Step 2: Requirement Analysis/Gathering Requirements of Splitwise
- Step 3: Low-Level Design (LLD) of Splitwise
- Scaling Splitwise- How It Is Achievable Invoking Caching

About Expense-Sharing Applications

AD

Expense-sharing applications are used to log, track, and monitor the expenses of users with each other or in a group. The most famous example of such an application is Splitwise.

About Splitwise

eSplitwise is an app available on Android and IOS that helps users track their expenses. Users can split their expenses for trips, movies, taxis, or any other day-to-day expenses between family, roommates, and friends. Users just need to enter the expense in the system, Splitwise simplifies and tracks the overall settlement status. When a user enters the details of the expense, the app gives a choice of type of split the user wants like exact, equal, or percentage. The app keeps the history of all expenses and allows to record a payment. The app also allows users to make a group and track expenses in that group.

- '**Simplify debts**' is a feature of Splitwise that restructures debt within groups of people. It does not change the total amount that anyone owes, but

AD

it makes it easier to pay people back by minimizing the total number of payments.
- Along with it, Spliwise Pro has paid features like users can scan receipts to record a payment, analyze their expenses with charts/graphs, can make settlement payments through the app

How do Expense-Sharing applications work?

- Suppose there are 3 flatmates sharing a flat. Now electricity, daily utility expenses, rent, and many other expenses. Consider rent of 12000.
- If any user pays the rent, it should be distributed equally among all users. Hence 12000/3 = 4000 is lent by the rest of the users(flatmates).

Here Splitwise will do the final math and indicate the settlement that the users have to make. We just have to build an expense-tracking application like this.

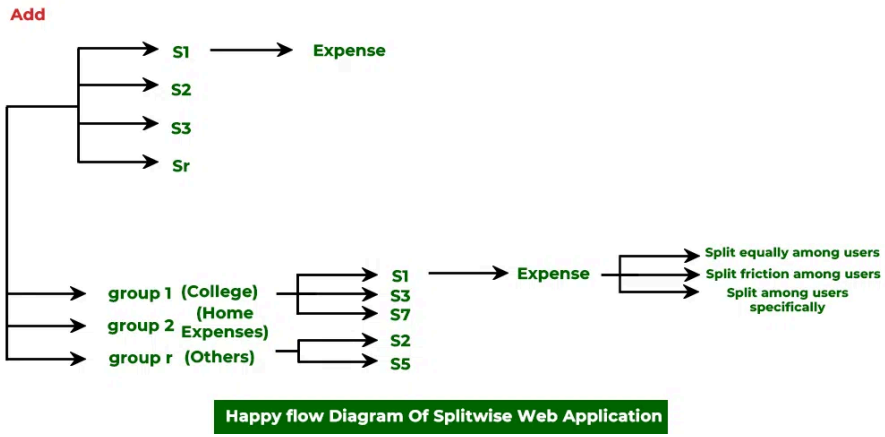
System Design of Splitwise

Step 1: Defining Happy Flow for Splitwise

Let us first define a happy flow corresponding to which we will easily be able to plot out requirement

gathering:

AD



Happy Flow Diagram of Splitwise Web Application

Note: We will be discussing the importance of this happy flow later while designing the low-level design of the system.

Step 2: Requirement Analysis/Gathering Requirements of Splitwise

Prioritized Requirements:

- Users can **add expenses.**
- Users can **edit expenses.**
- Users can delete an existing expense.
- Users can **settle their expenses.**
- Users should be allowed to make **groups** and add, edit, and settle expenses in the group.

- Users can **list all the expenses** that he has done with a specific friend.
- All involved users in an expense **should be notified** when the expense is created, edited, or deleted.

Design Requirements:

- User: Every individual will have an ID, name, email, and phone number.
- Expense: This could be outside the group and within the group.
    - **EQUAL**: Expense is distributed among all users. Example: Rent among users.
    - **EXACT**: Expense is distributed among specific users but certainly not all.
    - **PERCENTAGE**: Expense is distributed in accordance with proportions among users. Example: Road trip expense.

Listing out non-required system Gatherings

Non-required system requirements are as follows:

- **No need to maintain records for the activity:** We do not need to maintain an **activity log** for every event as we are making sure consistency is

AD

achieved throughout while one user is adding expenses another activity for the same time instance is stopped, hence we do not need to maintain a log for the same.

- **Comments for records**
- **Authentication service:** Nor do we require authentication to add a user to a group/outside the group.

Step 3: Low-Level Design (LLD) of Splitwise

3.1: Defining Objects | Splitwise LLD

Here we will be having below objects that are as follows:

- **User object:** This class consists tells how every user is defined.

```
class User {

    // User default id, with user profile image
and their
    // bio.
    userID uid;
    string ImageURI;
```



```
    string bio;
}
```

AD

- **Group object:** Now let us define the class for the group object.

```
class Group {
    // Group id of this group
    GroupID gid;

    // List of all users
    List<User> users;
    // image of the group
    string ImageURI;
    // title of the group
    string title;
    // group description
    string description;
}
```

- **Expense object:** These objects must map each user to their balance.

```
class Expense {
    ExpenseID expenseId;
    bool isSettled = false;
    // flag to determine whether this expense is
settled or
    // not.

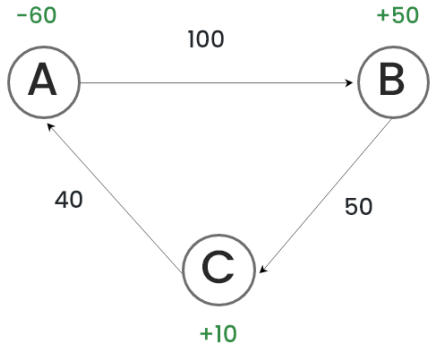
    // Creating map of Users and their balances
    map<User, Balance>;

    GroupID gid; // This expense belongs to which
group -
                 // optional
```

AD

```
    // metadata
    string title;
    int timestamp;
    string imageURI;
}
```

- **Balance object:** There are 2 types of users:
    - Users who are receiving the money. They owe. They have a positive balance.
    - Users who are paying the money. They lent. They have a negative balance.



Splitwise Diagram

Balance object

```
class Balance {
    string currency; // Note: Here we are bound
that all
                     // transactions are
happening across
                     // same currency.
    long amount; // We are using long as expense
reported
```

```
                    // can be large.
}
```

AD

3.2: System Behavior Definition | Splitwise LLD

- **Add Expense:** We have user and balance objects, and then we can persist them in the database.
- **Edit Expense:** Each expense possesses a unique ID and we can use the ID to change mappings or other metadata.
- **Settle Expense:** We will make **isSettled** to true. We will use the balancing algorithm to settle the expenses.
- **Add, settle, and edit expenses in a group:** Each expense object has a groupID. So we can add, settle and edit expenses in a group.

3.3: Generating a Problem Statement | Splitwise LLD

After glancing through the above objects and behavioral characteristics, the Problem Statement generated is as follows:

> ***Input:*** *Input given to us is a list of transcations as* ***transactions[from, to, transAmount]***

Upcoming Courses

Output: *Number of minimum transactions to settle the debt*

Sample Example: ***[*** *User2, User1, 100K* ***],*** ***[*** *User 1, User2, 80K* ***],*** ***[*** *User1, User2, 20K* ***]***

Explanation:

- The first transaction occurs, here User2 gives 100 K to User 1 which means User2 owes 100K to User2 and User1 lent 100K.
- Again a transaction is made where- User2 repays to User1 a sum of 80K which means now User1 only lent 20K.
- Again, a transaction occurs in which User2 pays 20K to User1, meaning nobody owes anyone's money. It means the accounting is set up.

We could have done the transaction in a lesser number to settle the debt which indeed is the goal of our system.

Consider the below graph in order to examine the net change in cash of each person in the group which can be determined by using the below formula:

$$NetChangeInCash = [SumOfCashInflow] - [SumOfCashOutflow]$$

MERN Full Stack Cour...
Starting from - October 24, 2026
• LIVE
4.7

Data Science 360...
Starting from - October 25, 2026
• LIVE
4.7

AI Engineering Cours...
Starting from - October 31, 2026
• LIVE
4.7

View All →

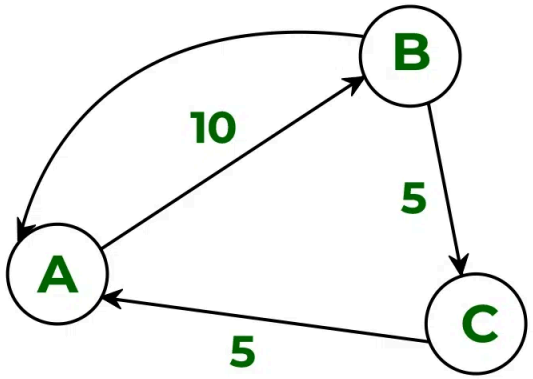
Fresher Jobs Experienced Jobs

Cybersecurity intern
CyArt Tech
Internship • Remote -
Apply by Sat Oct 31 2026

ServiceNow Developer
ITnow Inc
Fresher • Remote -
Apply by Sun Nov 01 2026

AI/ML Intern
PGAGI Consultancy Pvt Ltd
Upto 1 Years • Remote -
Apply by Mon Nov 30 2026

View All →



Net Change in Cash(A) = (Sum of Cash Inflow(A) — Sum of Cash Outflow(A))
= (5) - (10)
= -5

As depicted from the above graph, A has a net change in cash of 5 rupees, whereas A lent 4 rupees as there is a negative sign representing debt. Similarly, if we do calculate cash flow for node C(user C) it is as follows:

Net Change in Cash(C) = (5) - (5) = 0
It means user C is already settled up.

Now we are good to go with defining Node.java and paymentMode.java after having a good understanding:

```
class Node {
    UserID user_ID;
    int final_balance;
}
```

Now let us also define a class for **PaymentMode** as seen above in the illustration:

```
class PaymentNode {

    // Specific unique ID- that is our UserID
    UserID from;
    UserID to;

    // It is the amount that is been transefered
    long amount;
}
```

3.6: Complete LLD Implementation of Expense Sharing Application - Splitwise

> *Low-level design of a Splitwise system can be implemented with the usage of heap structure. Here, Nodes in graphs represent 'Users' and edges represent 'transaction amount'*

Pseudocode:

```
List<PaymentNode> makePaymentGraph()
{

    // Step 1: Let us first store the absoulte
value
    Max_Heap<Node> type1;
    Max_Heap<Node> type2;

    List<PaymentNode> graph;

    // The sum of balance of all users always
```

```
results in 0
    // so if first heap is empty then
    // second heap will also have no elements.
    loop(type1.isNotEmpty())
    {
        // Accessing element at the top of
container
        receiver = type1.top();
        sender = type2.top();

        // Step 2: Removing the element
        // using pop() method
        type1.pop();
        type2.pop();

        // Step 3: Getting the amount transferred
by getting
        // minimum of sender and receiver final
balance
        // As we need minimal edges
        amountTransferred =
min(sender.finalBalance,

receiver.finalBalance);

        // Step 4: Here we are creating the graph
as
        // discussed in illustration
        graph.insert(PaymentNode(sender.uid,
receiver.uid,

amountTransferred));

        // Step 5: Settling the balance as amount
is being
        // trasfered
        sender.finalBalance -= amountTransfered;
        receiver.finalBalance -=
amountTransfered;

        // Step 6: Checking for the edge cases
        if (sender.finalBalance != 0)
            type2.push(sender);
```

```
        if (receiver.finalBalance != 0)
            type1.push(receiver);
    }

    // Step 7: Returning the payment graph
    return PaymentGraph;
}
```

Now we can propose out API design which is as follows:

3.7: API Design:

> ***Users --> Balance --> Expense --> Group --> Payment***

User *getUser(UserID user_id)* ***User[]*** *getUsersInGroup(GroupID group_ID)*
ExpenseID *addExpense(GroupID group_id, UserID user_id, long amount, string currency)* ***ExpenseID*** *editExpense(GroupID group_id, UserID user_id, long amount, string currency)* *void settleExpense(ExpenseID expense_id)* ***Expense*** *getExpense(ExpenseID expense_id)* ***ExpenseID []*** *getGroupExpenses(Group group_id)*

GroupID *makeGroup(Users[] users, string name, string imageURI, string description)* ***GroupID*** *getGroup(GroupID)*
PaymentGraph *getGroupPaymentGraph(GroupID group_ID)*

Conclusion: Splitwise can be further scaled by invoking the concept of caching as the in-real time we are making it

Scaling Splitwise- How It Is Achievable Invoking Caching

Scaling via Invoking Data Inconsistency:

We need to take care to maintain consistency and synchronization of the system. It is because just like in designing ATM machines we need to keep a stronghold that any transactions or operations should not be let go or get duplicated as created.

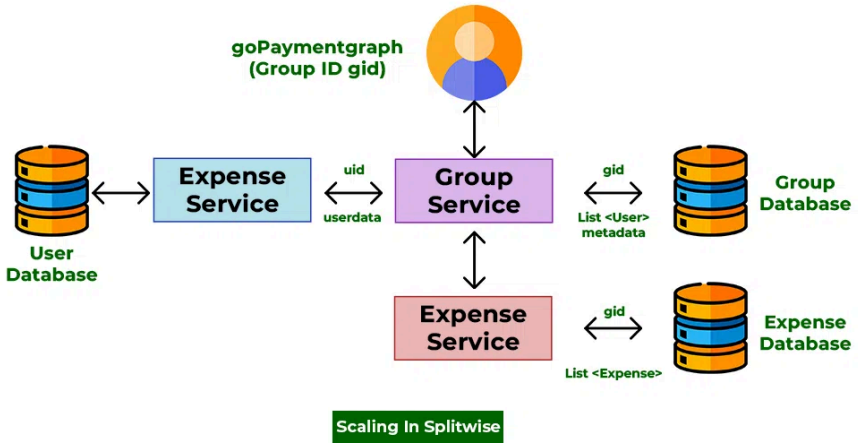
Suppose A transfers all of its available funds to B. Ofcource's combined balance remains the same, it just transfers from one account to another and is therefore

being tracked via this app. But what if during this write operation, there is another read operation over A at the same time, and the value overwrites with some other value or any garbage value or 0? Now our data becomes inconsistent. Hence in order to solve this problem we do have 2 solutions with us:

1. **Read Locks and Write Locks:**
   - Put read lock when updating expenses While updating the expense objects we will not allow users to read data. It will make sure that the data users get will be consistent.
   - A write lock ensures that no other thread can acquire a read or write lock while the write operation is in progress. This is crucial for maintaining data consistency, especially during critical operations such as transferring funds.

2. **Making objects mutable:** Make objects immutable So whenever there is an edit request we will create a new object and update data in a new object. Once the update is completed we will point the reference to the new object. Users might get old data however we can be sure that data will be

consistent. This will make our system eventually consistent.



Scaling (Actual Splitwise Application)

If we look carefully at API then look at these 2 APIs as listed below:

PaymentGraph getGroupPaymentGraph()
Expenses[] getGroupExpenses(GroupID groupo_id)

Comment

S **solanki...** 0

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Careers
Contact Us
Corporate Solution
Campus Training
Gr

Explore
POTD
Practice Problems
Blogs
Upskill Courses
Connect

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
GATE
School Subjects
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Data Science Programming Languages
DevOps & Cloud
GATE
MongoDB Certifications

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

GET IT ON Google Play
Download on the App Store

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved